

Mitigazione di Attacchi Botnet Stealth

Application-Layer tramite Analisi MIR e SDN

Project Work in Networks and Cloud Infrastructures (NCIS)

Angelo Carrillo

Corso di Laurea Magistrale in Ingegneria Informatica

Università degli Studi di Napoli Federico II

Napoli, Italia

a.carrillo@studenti.unina.it

Sommario—Gli attacchi botnet a livello di applicazione di tipo *stealth* sono una minaccia sempre più importante e critica per le infrastrutture di rete moderne; essi, operando con tassi di traffico contenuti e simulando il comportamento genuino di un utente umano, riescono a superare i tradizionali controlli implementati dai sistemi con soglie volumetriche statiche. Il presente lavoro sfrutta un’architettura modulare mediante OpenFlow e un controller Ryu, così da rilevare e mitigare la suddetta classe di attacchi mediante *Deep Packet Inspection* (DPI) e calcolo dinamico del *Message Innovation Rate* (MIR). In particolare, il MIR quantifica la diversità delle richieste HTTP degli host in una precisa finestra temporale; gli utenti legittimi producono valori prossimi a un MIR di 1.0, potendo usare un dizionario di URL molto vasto, mentre i bot sfruttano un catalogo ristretto che produce valori di MIR molto bassi. La struttura del sistema si divide in tre moduli, ovvero Monitor, Analyzer ed Enforcer, con l’integrazione di una REST API per controllare la lista dei blocchi. Il sistema permette quindi di superare sette criticità riscontrate nell’analisi del precedente progetto, come l’*over-blocking*, l’uso di soglie statiche e la rigidità delle decisioni di blocco centralizzate. I risultati empirici nei test hanno mostrato dei valori di $\eta_{bot} = 1.0$ e $\eta_{nor} = 0.0$, ovvero i bot vengono bloccati senza incorrere in falsi positivi.

Index Terms—SDN, Ryu, Botnet, MIR, OpenFlow, Cybersecurity, Mininet

I. INTRODUZIONE E THREAT MODEL

A. Contesto

I moderni attacchi DDoS vengono effettuati non più a livello di rete, mediante la saturazione di banda, ma vengono portati a termine al livello applicativo, ovvero al Layer 7. Un contatore di pacchetti è molto efficace nel rilevare i tradizionali attacchi volumetrici, ma fallisce totalmente con gli attacchi botnet *stealth*. Infatti, questi ultimi non saturano la banda, ma inviano richieste HTTP con una tempificazione simile a quella del traffico umano e un contenuto applicativo fortemente ripetitivo. Il loro obiettivo è quindi quello di esaurire le risorse dei server, come thread, connessioni, memoria, e senza particolare attenzione alla banda.

B. Threat Model

Si assume un modello di minaccia caratterizzato da bot distribuiti su diversi nodi *Edge* della topologia. L’orchestrazione di tali host da parte di un server *Command & Control* esterno è assunta, ma considerata fuori dal perimetro di modellazione. Il punto chiave dell’attacco *stealth* risiede

nell’utilizzo di un *dizionario di emulazione* molto ridotto, dato che i bot generano traffico mirato esclusivamente verso tre endpoint applicativi fissi: `/index.html`, `/api/v1/ping` e `/wp-login.php`. Gli utenti legittimi, invece, esplorano un catalogo composto da 100 percorsi unici. L’asimmetria nella varietà semantica del contenuto applicativo è il punto di snodo sulla distinzione che permette il rilevamento.

C. Il Vantaggio del Paradigma SDN

Il paradigma *Software-Defined Networking* porta a vantaggi fondamentali perché permette il disaccoppiamento tra *control plane* e *data plane*, che dona al controller SDN una visibilità centralizzata e globale su tutti i flussi di rete. Questa caratteristica consente di analizzare il traffico e reagire in tempo reale, mediante l’uso di regole di mitigazione OpenFlow direttamente sugli switch *Edge* più prossimi alla sorgente dell’attacco. Ciò impedisce al traffico malevolo di raggiungere il server target, bloccando la minaccia all’origine senza la necessità di riconfigurazioni manuali dei router o dei firewall.

II. RISOLUZIONE DELLE CRITICITÀ ARCHITETTURALI

L’architettura proposta nell’elaborato supera i limiti dei classici sistemi di difesa monolitici, affrontando sette specifici difetti implementativi mediante una filosofia basata su astrazione e disaccoppiamento.

A. Modularità e Separazione delle Responsabilità

La mancanza di modularità è superata con la scomposizione del controller in tre classi con responsabilità singola:

- **Monitor:** Gestisce gli eventi `Packet-In` OpenFlow, implementa il *learning switch* L2 ed esegue la *Deep Packet Inspection* (DPI) esclusivamente sullo switch *Edge* a cui è connesso il server (`dpid=4`), estraendo la *request-line* HTTP. Non ricorre mai all’implementazione di regole di *drop*.
- **Analyzer:** Eseguito su un thread indipendente tramite `hub.spawn` con periodo $T = 10s$. Viene sviluppato usando un blocco `try/except` che garantisce *fault tolerance* in caso di eccezioni a *runtime*. Ha il solo compito di calcolare il MIR e di delegare successivamente le azioni di mitigazione e blocco all’Enforcer.

- **Enforcer:** Costituisce l'unico punto del sistema che può costruire e inviare messaggi OpenFlow verso gli switch. Gestisce il dizionario `self.datapaths` e la `blocklist`, proteggendoli mediante `hub.Semaphore`. Questo approccio previene *race conditions*, dato che i tre thread concorrenti (`Packet-In`, `Analyzer` e `WSGI` per la REST API) vi accedono in modo simultaneo.

B. Granularità del Blocco e Prevenzione Over-blocking

Al fine di evitare il blocco indiscriminato di traffico legittimo e di ridurre la sensibilità topologica, le regole di mitigazione (`priority=200`) effettuano il *match* sull'indirizzo `ipv4_src` del singolo host compromesso, invece di effettuarlo sull'intera porta dello switch fisico, come nel caso del Piracy Shield italiano. La topologia di rete isola i bot su rami *Edge* distinti, come ad esempio `h5` su `s6`, `h6` su `s7`, garantendo la capacità dell'infrastruttura di individuare lo switch corretto indipendentemente dal percorso nell'albero. Inoltre, la *patch* proattiva HTTP (`priority=100`, *match* su `tcp_dst=80` e `ipv4_dst=SERVER_IP`) è installata una sola volta su `dpid=4` al momento dell'handshake con lo switch. Questo approccio risolve definitivamente le ridondanze architetturali che generavano regole sovrapposte in caso di medesime priorità, permettendo un comportamento deterministico senza dover ricorrere al flag `OFPFF_CHECK_OVERLAP`.

C. Flessibilità, Auto-ripristino e Controllo Esterno

La rigidità delle policy di blocco e sblocco viene superata attraverso due meccanismi complementari. L'impiego del parametro `hard_timeout=60s` richiede agli switch l'auto-ripristino passivo, permettendo il decadimento autonomo della regola di blocco e alleggerendo il carico computazionale del controller. Allo stesso tempo, un'interfaccia REST API garantisce un controllo attivo: l'endpoint `DELETE /firewall/block/{ip}` permette lo sblocco anticipato (manuale o automatizzato) senza attendere lo scadere del timeout, fornendo quindi un meccanismo di ripristino immediato in caso di falsi positivi. L'endpoint `POST /firewall/block` permette invece a entità di sicurezza esterne all'ecosistema SDN di contribuire alla *blocklist*. Infine, l'architettura supera l'accentramento delle policy: sia le rilevazioni interne dell'Analyzer, sia le chiamate REST esterne, invocano gli stessi metodi esposti dall'Enforcer (`block_ip` / `unblock_ip`), fornendo la stessa importanza operativa a tutte le sorgenti di intelligence.

III. IL MOTORE DI RILEVAMENTO (ALGORITMO MIR)

Il punto focale del sistema di rilevamento semantico è il *Message Innovation Rate* (MIR), una metrica che quantifica la varietà delle richieste applicative generate da un host.

Per ogni host sorgente, all'interno di una finestra temporale $T = 10s$, l'Analyzer registra il numero totale di richieste HTTP (N) e il numero di *request-line* testuali distinte (U) intercettate. Il MIR locale è definito come:

$$MIR = \frac{U}{N} \quad (1)$$

A parità di volume di traffico, ad esempio ≈ 1 req/s, utenti legittimi e bot risultano indistinguibili ai tradizionali sistemi di difesa volumetrici. Il MIR supera questa criticità risultando indipendente dal tasso di traffico: un bot con un *Emulation Dictionary* ristretto a 3 URL, pur simulando un attacco *slow-rate*, manterrà un rapporto U/N costantemente basso, ad esempio $MIR \approx 0.23$. Invece, un utente legittimo che naviga con un catalogo più ampio con 100 URL tenderà in modo asintotico a un $MIR \approx 1.0$.

A. Soglia Adattiva e Validità Statistica

Per superare l'inefficacia delle soglie statiche nel tempo, il sistema calcola dinamicamente il $MIR_{globale}$ sfruttando la media aritmetica dei MIR locali di tutti gli host attivi nella stessa finestra temporale. In particolare, un host viene classificato come botnet se e solo se soddisfa simultaneamente due condizioni:

- $N > 10$: garantisce la robustezza statistica del campione, prevenendo falsi positivi su host con bassissimo traffico o con traffico temporalmente raro;
- $MIR_{locale} < (MIR_{globale} - 0.3)$: confronta il comportamento del singolo nodo con il riferimento della totalità della rete. Se l'eterogeneità del traffico complessivo si riduce, la soglia si adatta in modo dinamico, come confermato dai risultati riportati nella Sezione V.

B. Ottimizzazione della Deep Packet Inspection

L'ispezione DPI è topologicamente limitata in modo esclusivo allo switch *Edge* del server (`dpid=4`). In caso di ispezione del *payload* ad ogni *hop*, un singolo pacchetto in transito attraverso l'intera infrastruttura arriverebbe a generare molteplici eventi `Packet-In`, causando un'inevitabile alterazione della misurazione della variabile N con un fattore moltiplicativo artificiale. Di conseguenza, sugli switch di transito, il traffico HTTP è instradato con un approccio proattivo verso il server tramite singole *flow entry* dedicate (`priority=10`), evitando il sovraccarico del *control plane* dell'architettura SDN.

IV. SETUP SPERIMENTALE

A. Topologia Emulata

L'infrastruttura di test è stata emulata utilizzando l'ambiente Mininet con nodi Open vSwitch (OVS) configurati per il protocollo OpenFlow 1.3. La topologia è composta da una struttura ad albero a tre livelli: uno switch *Core* (`s1`), due switch di *Aggregation* (`s2`, `s3`) e quattro switch *Edge* (`s4`, `s5`, `s6`, `s7`), connettendo un totale di sei host. I Datapath ID (`dpid`) sono stati assegnati in modo statico per garantire la perfetta corrispondenza con la mappa di associazione del controller. I ruoli operativi sono stati distribuiti strategicamente: il server target (`h1`) e una parte dell'utenza legittima insistono su `s4`, mentre gli altri utenti legittimi e i bot (`h5`, `h6`) sono isolati su rami *Edge* distinti nella topologia (`s6`, `s7`). La separazione garantisce l'efficacia del sistema di mitigazione indipendentemente dal percorso di rete.

B. Generazione del Traffico

L'iniezione del traffico applicativo è orchestrata da un singolo script unificato (`traffic_generator.py`), il cui comportamento varia in base al parametro di lancio `--role`. I nodi bot simulano l'attacco ricorrendo a un *Emulation Dictionary* ristretto a soli 3 URL fissi. Gli utenti legittimi, al contrario, generano le richieste attingendo da un catalogo di 100 percorsi generati combinando categorie, ID numerici e suffissi casuali. Al fine di garantire l'indistinguibilità volumetrica delle due tipologie di host, entrambi i ruoli implementano intervalli temporali tra una richiesta e la successiva distribuiti casualmente tra 0.4 e 1.5 secondi.

C. Cattura e Analisi Forense

Al fine di ottenere una validazione a livello di pacchetto, il tracciamento del traffico è integrato direttamente nel modulo di avvio `topology.py`. Un'istanza di `tshark` viene eseguita in modo automatico mediante la primitiva `h1.popen()` all'interno del *namespace* di rete isolato del server target, sfruttando un filtro BPF `tcp port 80` sull'interfaccia `h1-eth0`. L'integrità del file di cattura è permessa da un blocco `finally` che, all'uscita della CLI, invia un segnale `SIGTERM` al processo `tshark`, forzando il *flush* dei buffer e la scrittura del *trailer* .pcap prima che avvenga la terminazione.

V. VALUTAZIONE DEI RISULTATI

I test empirici hanno dimostrato l'efficacia dell'approccio proposto. La valutazione si basa sull'analisi incrociata dei tracciati forensi e delle metriche aggregate dal controller SDN.

A. Analisi Semantica e Andamento MIR

La Figura 1 evidenzia il tracciato di cattura Wire-shark, mostrando in un formato visivo il comportamento asimmetrico degli host. Mentre gli utenti legittimi, come 10.0.0.2 e 10.0.0.3, necessitano di percorsi sempre diversi, i nodi compromessi, come 10.0.0.5 e 10.0.0.6, operano la reiterazione ciclica dello stesso set di endpoint (`/index.html`, `/api/v1/ping`, `/wp-login.php`), esponendo inequivocabilmente la firma del loro *Emulation Dictionary*.

Questa asimmetria viene quantificata nel grafico dell'andamento MIR (Figura 2). Gli utenti legittimi mantengono un valore stabile di $MIR \approx 1.0$. Al contrario, dopo una inevitabile fase transitoria nelle primissime finestre a causa di un campione N ancora statisticamente non significativo, non appena il volume di richieste supera la soglia di validità ($N > 10$), il MIR dei bot decade in modo netto sotto il valore di 0.3. La soglia dinamica globale si adatta in modo costante, mantenendosi saldamente al di sopra del MIR dei bot (tipicamente tra 0.4 e 0.7).

B. Throughput, Mitigazione e Overhead

L'efficacia della frammentazione *Edge-based* è visibile nella Figura 3. Nel momento esatto in cui scatta il blocco, il *throughput* dei bot si azzerà a causa del *drop* dei pacchetti

No.	Time	Source	Destination	Protocol	Length	Info
446	19.483751753	10.0.0.5	10.0.0.1	HTTP	166	GET /wp-login.php HTTP/1.1
454	19.526436498	10.0.0.3	10.0.0.1	HTTP	168	GET /api/6391/v1vcj HTTP/1.1
461	19.682878958	10.0.0.6	10.0.0.1	HTTP	164	GET /index.html HTTP/1.1
471	19.922326397	10.0.0.4	10.0.0.1	HTTP	172	GET /support/1152/spisu HTTP/1.1
479	19.961356258	10.0.0.2	10.0.0.1	HTTP	172	GET /search/6318/lghax HTTP/1.1
487	20.205981080	10.0.0.5	10.0.0.1	HTTP	166	GET /wp-login.php HTTP/1.1
495	20.667406651	10.0.0.4	10.0.0.1	HTTP	169	GET /api/14476/xr1tb HTTP/1.1
503	20.786441209	10.0.0.3	10.0.0.1	HTTP	173	GET /support/96135/homdy HTTP/1.1
511	20.868949553	10.0.0.6	10.0.0.1	HTTP	165	GET /api/v1/ping HTTP/1.1
519	20.968054132	10.0.0.2	10.0.0.1	HTTP	170	GET /blog/17358/caduw HTTP/1.1
526	21.082935923	10.0.0.4	10.0.0.1	HTTP	170	GET /docs/19340/y1b1f HTTP/1.1
536	21.343415163	10.0.0.5	10.0.0.1	HTTP	164	GET /index.html HTTP/1.1
544	21.636095146	10.0.0.2	10.0.0.1	HTTP	173	GET /support/27388/pdhdg HTTP/1.1
552	21.893844979	10.0.0.5	10.0.0.1	HTTP	165	GET /api/v1/ping HTTP/1.1
560	21.944117448	10.0.0.6	10.0.0.1	HTTP	164	GET /index.html HTTP/1.1
568	22.075089909	10.0.0.4	10.0.0.1	HTTP	169	GET /api/76526/xdpcn HTTP/1.1
576	22.261525986	10.0.0.3	10.0.0.1	HTTP	174	GET /articles/98453/rezde HTTP/1.1
584	22.358310855	10.0.0.2	10.0.0.1	HTTP	172	GET /videos/21086/loybz HTTP/1.1
591	22.787087449	10.0.0.3	10.0.0.1	HTTP	171	GET /users/85754/xmtmm HTTP/1.1
601	22.883346870	10.0.0.5	10.0.0.1	HTTP	165	GET /api/v1/ping HTTP/1.1
609	23.191363962	10.0.0.2	10.0.0.1	HTTP	168	GET /api/4624/ytkhe HTTP/1.1
617	23.359112903	10.0.0.6	10.0.0.1	HTTP	164	GET /index.html HTTP/1.1
625	23.575455511	10.0.0.4	10.0.0.1	HTTP	171	GET /videos/2060/qzqhw HTTP/1.1
633	23.699262483	10.0.0.5	10.0.0.1	HTTP	164	GET /index.html HTTP/1.1
641	23.892620653	10.0.0.3	10.0.0.1	HTTP	172	GET /images/32783/vmaga HTTP/1.1
649	24.089779044	10.0.0.2	10.0.0.1	HTTP	174	GET /articles/99266/qwqcm HTTP/1.1
657	24.401227056	10.0.0.6	10.0.0.1	HTTP	164	GET /index.html HTTP/1.1
665	24.526215116	10.0.0.3	10.0.0.1	HTTP	174	GET /products/85383/zn1fv HTTP/1.1
673	24.897301522	10.0.0.5	10.0.0.1	HTTP	165	GET /api/v1/ping HTTP/1.1
681	24.993366359	10.0.0.4	10.0.0.1	HTTP	170	GET /blog/68433/gngta HTTP/1.1
689	25.294560802	10.0.0.2	10.0.0.1	HTTP	174	GET /products/79899/cnengs HTTP/1.1
697	25.375347109	10.0.0.6	10.0.0.1	HTTP	166	GET /wp-login.php HTTP/1.1
705	25.698358329	10.0.0.2	10.0.0.1	HTTP	174	GET /products/97649/eahhw HTTP/1.1
713	25.712987280	10.0.0.5	10.0.0.1	HTTP	165	GET /api/v1/ping HTTP/1.1
721	25.754024633	10.0.0.3	10.0.0.1	HTTP	174	GET /products/69071/zvfoo HTTP/1.1
729	26.058879189	10.0.0.4	10.0.0.1	HTTP	171	GET /videos/2473/nqpfm HTTP/1.1
737	26.378938308	10.0.0.6	10.0.0.1	HTTP	165	GET /api/v1/ping HTTP/1.1
745	26.518051436	10.0.0.3	10.0.0.1	HTTP	171	GET /users/34913/otlkc HTTP/1.1
753	26.757824623	10.0.0.2	10.0.0.1	HTTP	173	GET /support/54323/bxals HTTP/1.1
761	26.83212618	10.0.0.6	10.0.0.1	HTTP	164	GET /index.html HTTP/1.1
769	26.84791034	10.0.0.5	10.0.0.1	HTTP	165	GET /api/v1/ping HTTP/1.1
777	26.956726052	10.0.0.4	10.0.0.1	HTTP	174	GET /products/34913/otlkc HTTP/1.1
785	27.387258576	10.0.0.5	10.0.0.1	HTTP	165	GET /api/v1/ping HTTP/1.1
793	27.682483107	10.0.0.3	10.0.0.1	HTTP	172	GET /search/53655/aouls HTTP/1.1

Figura 1. Cattura tshark: evidenza dell'Emulation Dictionary del bot.

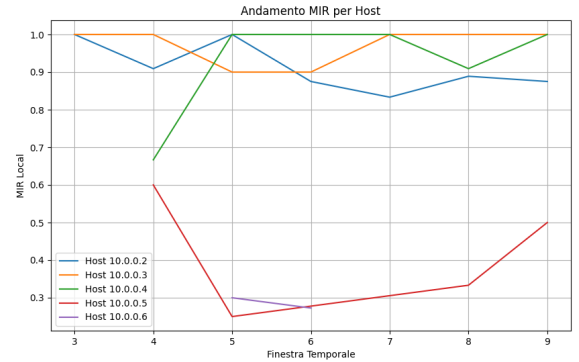


Figura 2. Andamento del MIR per host nel tempo. Si nota il drop del traffico bot.

malevoli, mentre il traffico degli utenti prosegue senza alcuna alterazione, mostrando l'assenza di falsi positivi o disservizi per l'utenza genuina in modo assoluto. Trascorsi i 60 secondi di `hard_timeout`, si nota un brevissimo picco di traffico bot, che viene immediatamente riconosciuto e mitigato dall'Analyzer nella finestra successiva.

Sotto il profilo delle prestazioni, l'ispezione DPI non introduce *bottleneck* architetturali (Figura 4). I dati raccolti, infatti, dimostrano una latenza media di elaborazione nell'ordine dei centesimi di millisecondo (≈ 0.01 ms), a conferma del fenomeno secondo il quale l'ispezione DPI limitata esclusivamente allo switch *Edge* del server non introduce *overhead* misurabili.

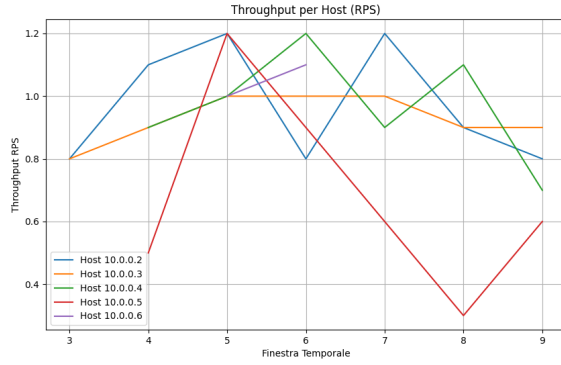


Figura 3. Throughput per host (RPS). Evidenza dell'intervento dell'Enforcer.

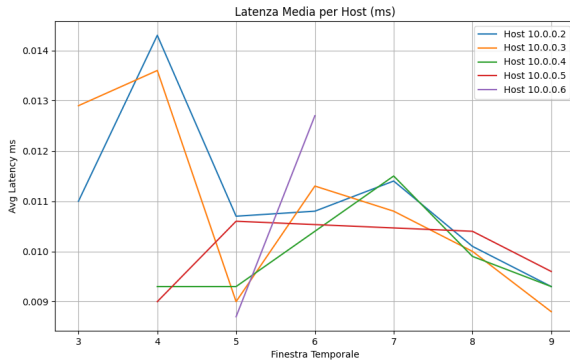


Figura 4. Latenza media di elaborazione DPI per host (ms).

C. Efficacia Complessiva del Sistema

Il framework del sistema soddisfa l'obiettivo architetturale in modo completo. A partire dalla sesta finestra temporale di osservazione, le metriche di accuratezza convergono ai valori ideali prefissati ($\eta_{bot} = 1.0$, ovvero individuazione e blocco totale dei nodi infetti, e $\eta_{mor} = 0.0$). Analizzando le finestre successive, si nota come la mitigazione venga demandata interamente all'hardware dello switch: infatti, scattato il blocco, il volume di traffico bot intercettato dal controller si azzerava ($N = 0$), dimostrando la totale eliminazione dell'*overhead* sul *control plane* fino al termine dello scenario di test.

VI. CONCLUSIONI E SVILUPPI FUTURI

L'elaborato dimostra come la rilevazione semantica basata sull'algoritmo MIR superi efficacemente i limiti delle difese volumetriche tradizionali nel contrasto agli attacchi *stealth application-layer*. Il sistema ha raggiunto un tasso di rilevamento perfetto ($\eta_{bot} = 1.0$, $\eta_{mor} = 0.0$), garantendo la totale assenza di falsi positivi.

Dal punto di vista architetturale, la scomposizione del paradigma monolitico in un approccio modulare (Monitor, Analyzer, Enforcer) ha permesso di risolvere le sette criticità identificate nell'analisi del precedente progetto. L'infrastruttura risulta quindi manutenibile, estensibile e *thread-safe*: la

gestione della concorrenza tramite `hub.Semaphore` garantisce la sincronizzazione corretta tra le elaborazioni asincrone dell'Analyzer e le interazioni esterne guidate dalla REST API.

Pur validando l'efficacia del modello, si evidenziano tre limitazioni da esplorare in sviluppi futuri. In primo luogo, la mappa di associazione tra host e switch *Edge* è attualmente statica; in un ambiente di produzione, essa richiederebbe un meccanismo di *discovery* dinamico, ad esempio mediante LLDP e `ryu.topology.api`. In secondo luogo, il calcolo del MIR si basa su finestre temporali di dimensione prefissata, esponendo il sistema a tecniche di elusione sincronizzate sui limiti della finestra, una vulnerabilità risolvibile adottando un approccio a finestre scorrevoli. Infine, l'attuale implementazione non contempla attacchi coordinati con un approccio *multi-vettore*, in cui i nodi compromessi potrebbero tentare di eludere l'ispezione alternando l'utilizzo di *payload* generati in modo procedurale al proprio *Emulation Dictionary*.